Variabili e tipi di dato

Lezione 3 — 1MAK, SST-INF-01c — Cos'è un programma (ripasso), variabili, char/int/float

29/09/2026

1 Ripasso: cos'è un programma

Definizione

Un **programma** è fondamentalmente una **lista ordinata di istruzioni** che diamo alla nostra macchina: una lista ordinata di operazioni da svolgere, eseguita **un passo alla volta**.

Esempio

L'analogia della ricetta: una **ricetta** è praticamente un programma, perché l'**ordine** delle operazioni conta – se faccio un'operazione prima di un'altra, cambia il risultato finale.
La **lista della spesa**, invece, **non** è un programma: se compro la carne prima della pasta, il risultato finale (la spesa fatta) non cambia – l'ordine in cui compro i singoli oggetti non ha importanza.

Nota

Un programma, alla fine, fa solamente **operazioni**: svolge **calcoli**, prende un oggetto (un dato) e lo trasforma in un altro.

1.1 Esempio pratico in aula: l'area del quadrato

Esempio

Il docente propone alla classe: *"calcola l'area del quadrato di lato 5"*. Un allievo, con il numero 5 "in mano", risponde correttamente **moltiplicando** 5 per 5 e ottenendo 25.
Domanda del docente: questo **5** che è stato consegnato all'allievo, dove viene tenuto? **In memoria**. E il programma, dove tiene i propri dati? Anche lui **in memoria**. Serve quindi creare una sorta di **scatola virtuale** in cui mettere il numero da **tenere a mente**.

Esempio

Le scatole "Carlo" e "Giuseppe" (esempio svolto alla lavagna): il docente crea una prima scatola e la chiama **Carlo**, e ci mette dentro il numero 5. Poi crea una seconda scatola, che chiama **Giuseppe**, e in questa scatola mette il **risultato** di **Carlo moltiplicato Carlo** (il **quadrato**, lato per lato).
Se in Carlo c'è il numero 5, in Giuseppe finisce il numero 25. Se in Carlo si mette il numero 2, in Giuseppe finisce il numero 4.
Questo è, in sostanza, il concetto di **variabile**: una scatola con un nome, che contiene un **valore** che il programma può usare e **modificare**.

2 Le variabili

Definizione

Le "scatole" nella memoria del computer, i cassetti in cui si tengono le **informazioni**, si chiamano **variabili**. Una **variabile** è un posto in cui si salva un'informazione. Le variabili possono essere **più o meno grandi**, in base al numero (o al **tipo di dato**) che devono contenere.

2.1 Quanta memoria ha un computer: dai terabyte ai byte

Esempio

Domanda in aula: quanta **memoria** (non RAM, l'archiviazione totale) ha il vostro computer di casa? Risposta della classe: circa **un terabyte (TB)**.
Il docente guida la classe nella conversione, passo per passo, ricordando che ogni volta che si passa da un'unità di misura alla successiva si aggiungono o si tolgono **sempre tre zeri**:

1 **TB**	=	1.000 **GB** (gigabyte)
1.000 GB	=	1.000.000 **MB** (megabyte)
1.000.000 MB	=	1.000.000.000 **KB** (kilobyte)
1.000.000.000 KB	=	1.000.000.000.000 **B** (byte)

Un terabyte corrisponde quindi a **mille miliardi di byte**: mille miliardi di "cassetti", ciascuno dei quali può **contenere un numero** che va, come vedremo, da 0 a 255.

Nota

Per la conversione tra un'unità di misura e quella successiva/precedente: andando verso l'alto (da byte verso terabyte) si **riducono gli zeri** e si cambia la sigla; andando verso il basso si **aggiungono zeri**. Il **fattore** è sempre 1.000 (tre zeri) tra un'unità e la successiva.

3 Il tipo `char`: la variabile più piccola

Definizione

Il **byte** è l'**unità base** di memoria: è composto da **8 bit**. La più piccola tipologia di variabile che si può creare in un programma si chiama **char**, ed è fatta da **8 bit (1 byte)**.
Con 8 bit si possono rappresentare $2^8 = 256$ **combinazioni** diverse, quindi una variabile `char` può memorizzare i numeri interi che vanno da **0 a 255**.

Attenzione

Perché 255 e non 256? Perché se si hanno 256 combinazioni possibili, e lo **0** è una di quelle combinazioni (rappresenta un numero valido, non "il vuoto"), il numero **più alto** raggiungibile è 255, non 256. Se serve memorizzare il numero **300**, una variabile `char` non basta più: serve un **tipo più grande**.

3.1 Ripasso: combinazioni e sistema binario

Esempio

Con due lampadine: quante **combinazioni** (accesa/spenta) sono possibili? **Quattro**: spenta-spenta, spenta-accesa, accesa-spenta, accesa-accesa.
Richiamando la **codifica binaria** vista nella lezione precedente, a ciascuna combinazione corrisponde un numero:

$$\begin{array}{ll} \text{spento-spento} & = 0 \\ \text{spento-acceso} & = 1 \\ \text{acceso-spento} & = 2 \\ \text{acceso-acceso} & = 3 \end{array}$$

Perché questa corrispondenza? Per la stessa regola di conversione decimale→binario vista nella lezione precedente: **dividendo ripetutamente per 2** e leggendo i resti **dal basso verso l'alto**. Ad esempio, per convertire il numero **2**: $2 \div 2 = 1$ con resto 0; $1 \div 2 = 0$ con resto 1. Leggendo i resti dal basso verso l'alto si ottiene **10**, cioè il 2 in binario.

Esempio

Con tre lampadine (3 bit), le **combinazioni possibili** salgono a $2^3 = 8$:

$$\begin{array}{ll} 000 & 0 \\ 001 & 1 \\ 010 & 2 \\ 011 & 3 \\ 100 & 4 \\ 101 & 5 \\ 110 & 6 \\ 111 & 7 \end{array}$$

Con **8 bit** (un char), le combinazioni possibili sono $2^8 =$ **256**: si va da 0 fino a $256 - 1 =$ **255**.

3.2 Dichiarare una variabile char

Esempio

```
char michele;
```

Questa riga **crea una variabile** di tipo `char`, chiamata `michele`, in grado di contenere un **numero intero** tra 0 e 255.

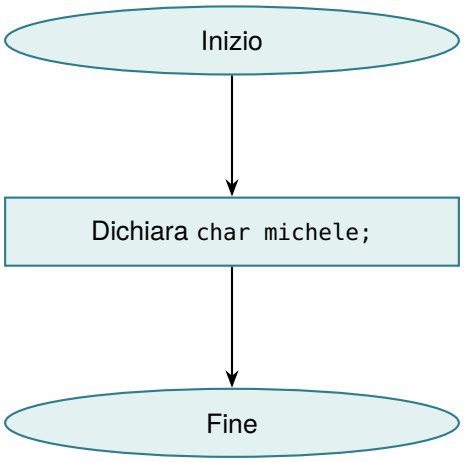


Figura 1: Diagramma di flusso: dichiarazione di una variabile `char`.

4 Il tipo `int`

Definizione

Esiste un'altra tipologia di variabile, chiamata **int** (*integer*, intero): occupa, come minimo, **16 bit**, cioè **2 byte**. Con 16 bit si possono rappresentare $2^{16} = 65.536$ combinazioni, quindi una variabile **`int`** può contenere numeri che vanno da **0 a 65.535**.

Esempio

Una variabile `int` può contenere numeri **molto più grandi** di un `char`: ad esempio il numero di abitanti del Friuli, la distanza in metri tra due città, l'altezza in metri di una montagna – qualunque valore esprimibile tra 0 e 65.535.

4.1 La dimensione di int dipende dalla macchina

Attenzione

La **dimensione** della variabile `int` **non è sempre uguale**: dipende dal **tipo di macchina** (dalla tecnologia dell'hardware) su cui gira il programma.

- Su un **Arduino**, che nasce con un microcontrollore a **8 bit**, l'`int` è comunque a **16 bit** (il minimo garantito dal linguaggio C).
- Su una macchina con un processore a **16 bit**, l'`int` resta a **16 bit**.
- Su macchine più potenti, a **32 o 64 bit** (ad esempio un computer con Windows a 64 bit), l'`int` prende esattamente la **larghezza della macchina**: quindi diventa a 32 o a 64 bit.

In sintesi: la dimensione della "scatola" `int` dipende dalla tecnologia con cui è costruita la macchina. Il minimo garantito è comunque sempre **16 bit**.

4.2 La scala dei numeri: da 0 a 65.535, e la variante con segno

Esempio

Disegno alla lavagna: il docente traccia una scala numerica da **0** a **65.535**, chiedendo alla classe di individuare il **punto medio**. La classe individua correttamente circa **32.768** come punto medio, e circa **16.000** come **un quarto** della scala.

Nota

Il linguaggio C, in realtà, normalmente **non parte da 0**: la scala ha la stessa ampiezza (sempre 65.536 numeri totali), ma invece di partire da 0, lo **0** viene messo a **metà scala**. Andando verso destra si sale fino a **+32.767**; andando verso sinistra (numeri negativi) si scende fino a **-32.768**. In questo modo, con una variabile `int`, si possono rappresentare numeri sia **positivi** che **negativi**.

Definizione

Riepilogando i **range** delle variabili viste finora:

- **char**: da **0** a **255** (**solo positivi**).
- **int**: da **-32.768** a **+32.767**.

Perché i due estremi non sono uguali in valore assoluto? Perché lo **zero** esiste **una sola volta** (non c'è un "più zero" e un "meno zero" distinti): quindi da una parte della scala (i positivi) bisogna **togliere un'unità** rispetto all'altra (i negativi), per non contare **due volte** lo zero.

Esempio

```
int paolo;
```

Questa riga **crea una variabile** di tipo `int`, chiamata `paolo`.

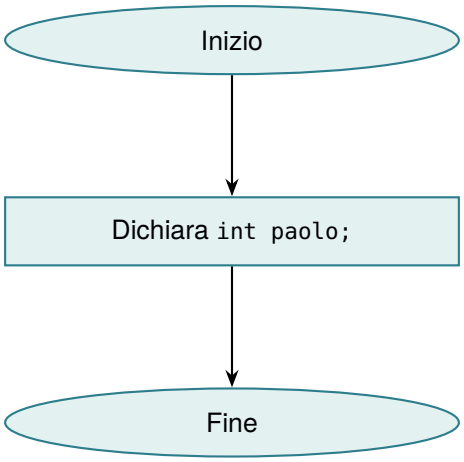


Figura 2: Diagramma di flusso: dichiarazione di una variabile `int`.

5 Numeri senza virgola e numeri con virgola

Attenzione

Sia il tipo `char` che il tipo `int` contengono **solo numeri interi, senza virgola**. Non è possibile memorizzare il numero 7,5 in una variabile `int`.
Se si prova a mettere il numero **32,9** in una variabile `int`, il valore memorizzato sarà semplicemente **32**: la parte decimale viene **persa**, **non arrotondata**. Lo stesso vale anche per 32,99999999: il risultato memorizzato resta 32. Questo comportamento, se non tenuto in considerazione, può **accumulare errori** nel tempo e diventare un problema serio in un programma.

6 Il tipo `float`

Definizione

Per memorizzare numeri **con la virgola** esiste una **terza tipologia** di variabile, chiamata **float**. Occupa **32 bit**, cioè **4 byte**.

Attenzione

Precisazione (non detta esplicitamente a lezione, verificata dopo): il **range reale** di un `float` a 32 bit **non** è "circa ± 2 miliardi" (quel valore è invece corretto per un `int`/`long` a 32 bit): il range effettivo di un `float` è **molto più ampio**, circa $\pm 3,4 \times 10^{38}$. Questo perché un numero in **virgola mobile** non usa tutti i suoi bit per contare in sequenza come un intero, ma li divide tra segno, **esponente** e **mantissa** (parte decimale), un po' come la **notazione scientifica**: questo permette di rappresentare sia numeri enormi che numeri piccolissimi, al costo di una **precisione limitata** sulle cifre significative.

Esempio

Esempio in aula: è possibile memorizzare in un `float` un numero come 322.423,7411? Sì, un `float` può contenere **numeri con la virgola** di questo tipo.

6.1 Perché scegliere il tipo giusto: l'analogia dei cassetti

Esempio

Analogia dell'armadio: un armadio grande ha tanti **cassetti**, alcuni grandi e altri piccoli. Se si ha un cassetto molto grande e ci si mette dentro solo un paio di calzini (perché non se ne possiedono altri), è uno **spreco di spazio**.
Allo stesso modo, in un programma: se bisogna manipolare **dati piccoli**, non ha senso occupare più spazio di memoria del necessario usando sempre il **tipo più grande**.

Esempio

Perché conta, in pratica: se un programma crea non una, ma **100.000 variabili**, e per ciascuna basterebbe un solo `byte` (`char`) invece di 4 byte (`float`), la **differenza** è di **300.000 byte** occupati inutilmente.
Esempio dei videogiochi: se un videogioco richiede una **scheda video** con 64 GB di RAM (già una spesa economica notevole), e il programmatore, non pensando a questi problemi, usasse sempre `float` anche dove non serve, la richiesta di memoria potrebbe salire fino a 500 GB di RAM. Una scheda video che oggi costa 1.000 euro, ne costerebbe allora 15.000: il gioco diventerebbe **ingiocabile** per la maggior parte delle persone, e il programmatore non lo venderebbe. È quindi interesse del **programmatore** mantenere lo spazio occupato dal programma **il più basso possibile**, per permettere al gioco di funzionare anche su hardware meno potente e costoso.

Definizione

Regola pratica per scegliere il tipo di variabile:

- Se il numero da memorizzare **ha la virgola**, serve **sempre** il `float` – anche se il numero è piccolo (ad esempio tra 1 e 5).
- Se il numero **non ha la virgola** ed è piccolo (ad esempio intorno a 200), basta un **`char`**: si risparmia memoria rispetto a un `int` o un `float`.
- Se il numero non ha la virgola ma è più grande di 255, serve un **`int`**.

Nota

Queste tre tipologie di variabili vanno **imparate a memoria**: per ciascuna bisogna sapere il numero di bit occupati (8, 16, 32) e il **range di numeri** rappresentabili. Non è sufficiente saperle riconoscere: vanno studiate.

7 Come si crea (dichiara) una variabile

Definizione

Per **creare una variabile** in C si scrivono, in ordine:

1. il **tipo** della variabile (`char`, `int` o `float`);
2. uno **spazio**;
3. il **nome** scelto per la variabile;
4. un **punto e virgola** (`;`) che chiude l'istruzione.

Esempio

Esempi scritti alla lavagna durante la lezione:

```
int paolo;
```

```
char michele;
```

Il **nome scelto** non deve necessariamente essere un nome di persona: si può chiamare una variabile `lampadina`, `sasso`, `zqw`, oppure semplicemente `x` – qualunque nome, purché rispetti le **regole** viste di seguito.

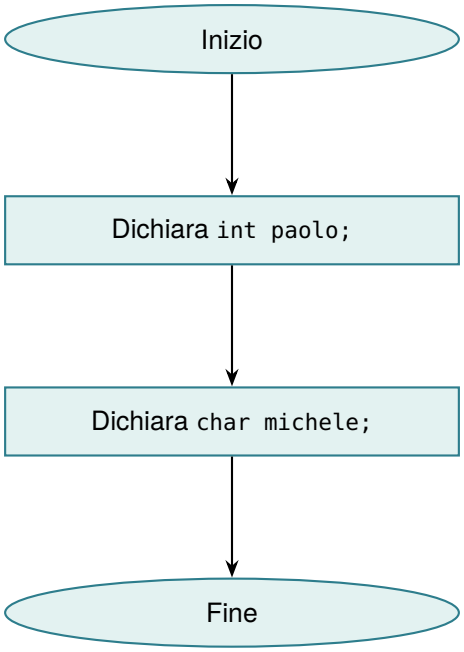


Figura 3: Diagramma di flusso: dichiarazione in sequenza di due variabili di tipo diverso.

8 Le regole per il nome di una variabile

Attenzione

Non si può dare a una variabile **qualunque** nome si voglia: il nome di una variabile deve rispettare **tre regole** precise.

8.1 Regola 1: non può iniziare con un numero

Esempio

```
int 7zq;   // NON valido: inizia con un numero
int zq7;   // valido: il numero non e' all'inizio
```

L'importante è che il nome **non inizi** con una **cifra**: un numero può comparire in qualunque altra posizione del nome, anche subito dopo la prima lettera.

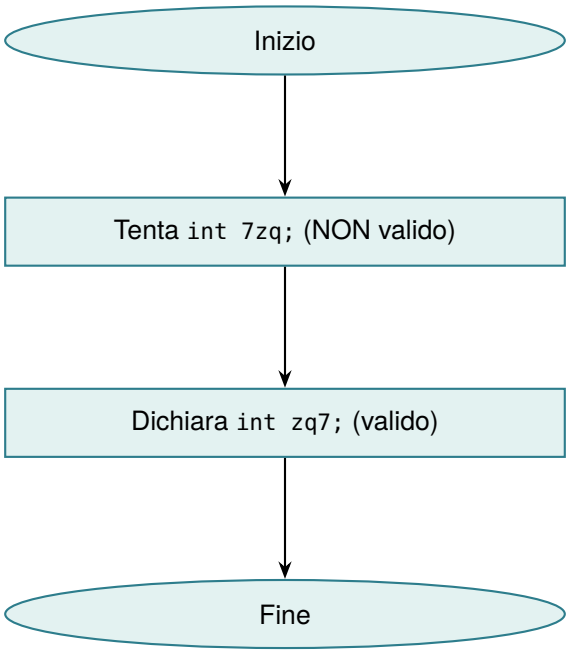


Figura 4: Diagramma di flusso: confronto tra un nome di variabile non valido e uno valido.

8.2 Regola 2: non può contenere simboli speciali

Definizione

Il nome di una variabile può contenere **solo lettere, numeri e underscore** (il trattino basso _, sulla tastiera in genere ottenuto con il tasto a destra del punto, tenendo premuto maiuscolo). **Nessun altro simbolo speciale** è ammesso: né +, $, €, /, \, apici, spazi, punti interrogativi, né alcun altro carattere non alfanumerico.

Esempio

Esempi scritti alla lavagna (validi e non validi):

```
int x+2;          // NON valido: contiene il simbolo '+'
int _x2;          // valido: underscore e' ammesso
int Ciao Mauro;   // NON valido: contiene uno spazio
int Giochi?;      // NON valido: contiene '?'
```

8.3 Regola 3: è case sensitive

Definizione

Il C è **case sensitive**: **maiuscole e minuscole** sono considerate **caratteri diversi**. Di conseguenza, due variabili che differiscono solo per maiuscole/minuscole sono due variabili **completamente distinte** (due "scatole" diverse), anche se si pronunciano allo stesso modo.

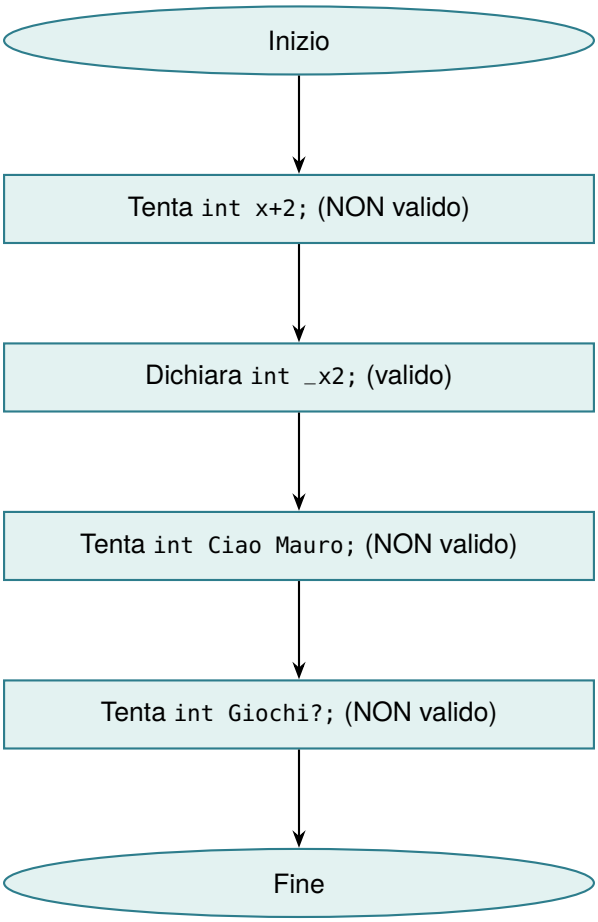


Figura 5: Diagramma di flusso: quattro tentativi di dichiarazione, con simboli validi e non validi nel nome.

Attenzione

Esempio in aula: una variabile chiamata `y` (minuscola) e una chiamata `Y` (maiuscola) sono **due variabili diverse**, anche se a voce si leggono nello stesso modo ("i greca"). Bisogna fare **molta attenzione** a questa distinzione: confondere le due può creare **errori difficili** da individuare in un programma.

Esempio

Ultimo esempio, lasciato in sospeso a fine lezione: **due variabili distinte**, in cui si può mettere ad esempio il numero 42 in una e 73 nell'altra – rispettando sempre le tre regole viste: **non iniziare con un numero**, e **non usare simboli** non ammessi.

9 Cosa ricordare di questa lezione

- Un **programma** è una **lista ordinata di istruzioni** eseguite un passo alla volta; **l'ordine conta** (a differenza di una lista della spesa).
- Una **variabile** è una "scatola" in **memoria** in cui si salva un dato, identificata da un **nome**.
- **char**: **8 bit** (1 byte), numeri interi da **0 a 255**.

- **int**: **minimo 16 bit** (2 byte, dipende dalla macchina), numeri interi da **-32.768 a +32.767**.
- **float**: **32 bit** (4 byte), numeri **con la virgola**, **range molto più ampio** di quanto la semplice dimensione in bit farebbe pensare.
- Scegliere sempre il tipo di variabile **più piccolo possibile** che sia sufficiente per il dato da memorizzare, per non **sprecare memoria**.
- Il nome di una variabile: non può **iniziare con un numero**, può contenere **solo lettere/numeri/underscore**, ed è **case sensitive**.